Color Image Processing

- Component Based Processing
  - Point operations
  - Linear operations
  - Non-linear operations
- Vector Based Processing
- Pseudocolor Methods
- Conclusion

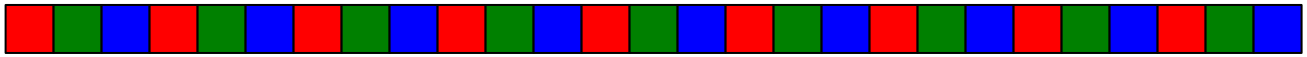
Credit: Prof. John Gauch

Component Based Processing

- One way to process color images is to separate the image into three component images and process each color channel independently
  - Advantage – simple to implement since we can reuse existing image processing algorithms/code
  - Disadvantage – may produce unexpected results for some images processing algorithms/images

Component Based Processing

- Packed versus unpacked color pixels
  - Packed – RGB values for pixels 0..N are stored in an interleaved format, typically one byte per component to create 24bit color pixels



  - Unpacked – RGB pixels are stored as three separate arrays of 0..N pixel values

Component Based Processing

- Packed and unpacked color images contain the same information, so the main difference is in memory *access* patterns
  - Processing a packed color image:
    - Unpack the 24bit color pixel to get components
    - Perform image processing on each component
    - Pack results back into 24bit color pixel
  - Processing an unpacked color image:
    - Perform image processing on each component

Component Based Processing

- The im_color class in our imaging system uses an unpacked color pixel representation with three im_short images

```
class im_color
{  public:
   im_short R;
   im_short G;
   im_short B;
… };
```

Component Based Processing

- Most of the image processing methods in the im_color class consist of three im_short method calls to process channels separately

```
void im_color::Interpolate(int xdim, int ydim)
{
   R. Interpolate(xdim, ydim);
   G. Interpolate(xdim, ydim);
   B. Interpolate(xdim, ydim);
}
```

Point Operations

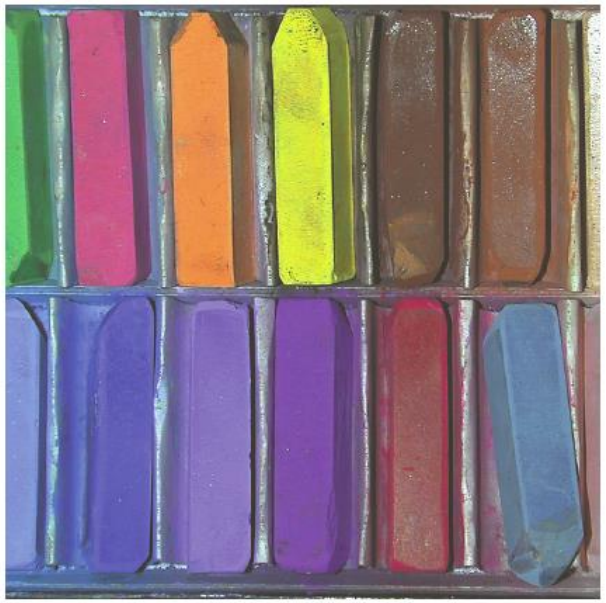
- Image brightness and contrast can be improved in color images by applying the *same* point operation to all three color channels

- We can use gamma correction, log scales or polynomial intensity mapping functions to implement the point operation

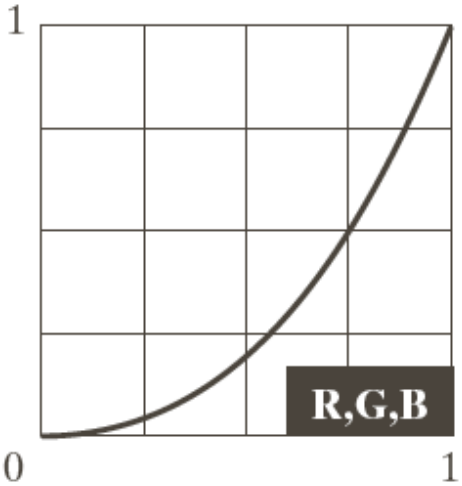
Point Operations

- Image too light – stretch histograms upwards

Light



Corrected

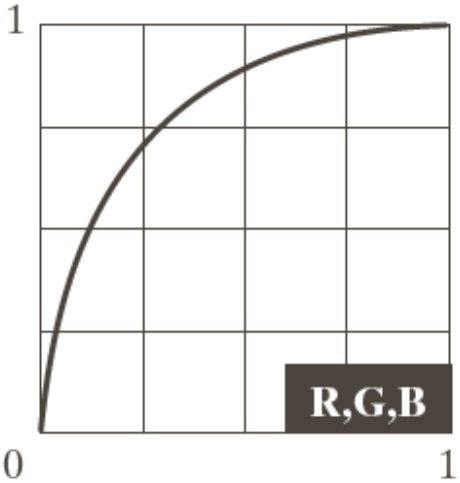


Point Operations

- Image too light – stretch histograms downwards

Dark

Corrected

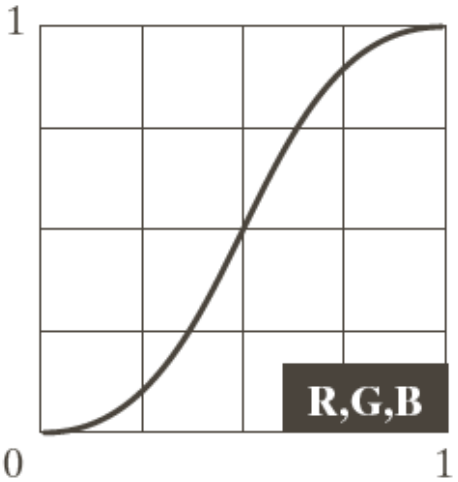


Point Operations

- Image flat – stretch histograms in both directions

Flat

Corrected



Point Operations

- The hues of colors in an image will change if the mapping functions are not the same in all three color channels

- Example: mapping pixels to [0..255] range prior to output will stretch one color more than the others if it has a smaller range

Point Operations

- Original image had R[55..253], G[30..255], B[0..253] so R was stretched much more than G or B

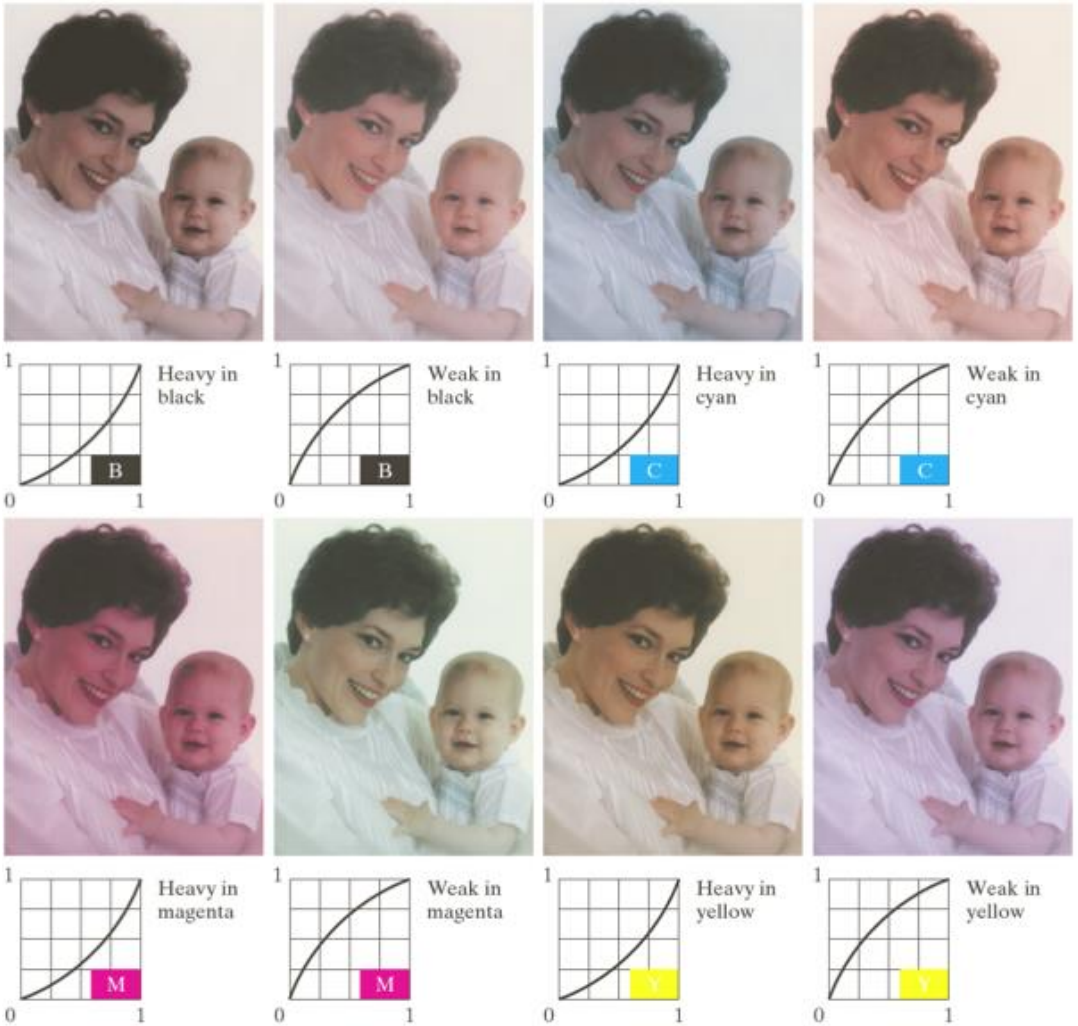
Point Operations

- Original image had R[6..251], G[1..221], B[1..108] so B was stretched much more than R or G

Point Operations

- Printers may require correction of one CMYK color component to restore balance

Original/Corrected



Point Operations

- Point operations are also used for inverting negatives that are digitized by film scanners in order to produce color images

- In this case, the mapping functions are:

$$R = 255 - R$$

$$G = 255 - G$$

$$B = 255 - B$$

Point Operations

negative

original

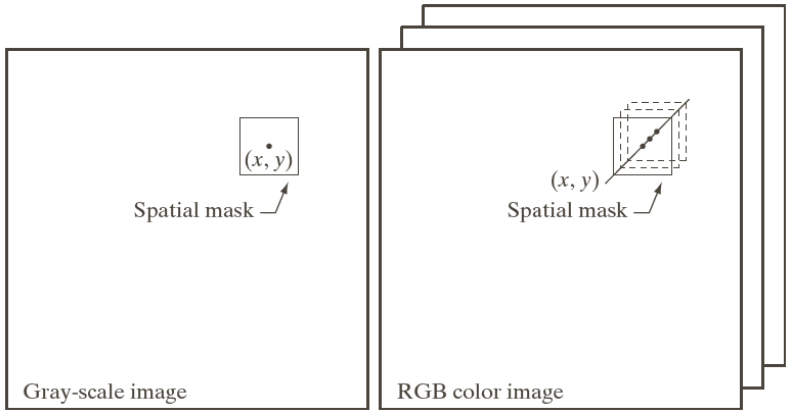
Point Operations

negative

original

Linear Operations

- Linear image processing can be used to smooth or sharpen color images by processing each color component separately in the spatial or frequency domain



Linear Operations

- Gaussian smoothing of RGB images will introduce colors that are *in between* adjacent color regions

original sigma=2 sigma=5

Linear Operations

- In most cases, Gaussian smoothed RGB images will look natural to our eye

original

sigma=2

sigma=5

Linear Operations

- Gaussian smoothing in HSI color space will produce unexpected/strange results

original

sigma=2

sigma=5

Linear Operations

- Unsharp masking can be applied to RGB images to enhance visibility of details

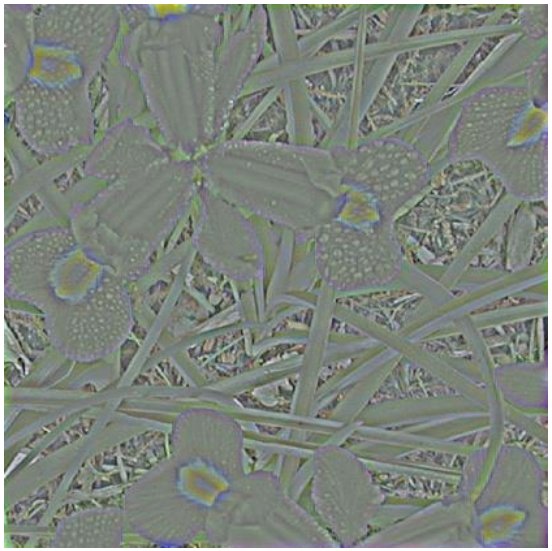
original n=10, w=0.9 n=100, w=0.9

Linear Operations

- Sometimes highpass and lowpass filters will introduce unintended color changes

original

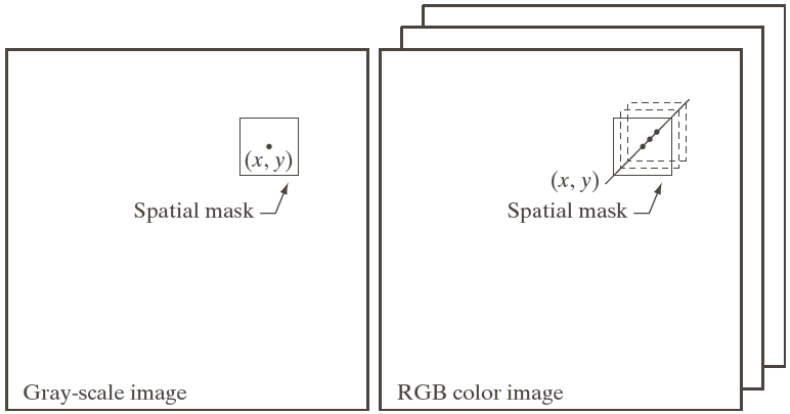
Butterworth low pass 20



Butterworth high pass 20

Non-linear Operations

- Non-linear image processing can also be used to smooth or sharpen color images by processing each component separately in the spatial or frequency domain



Non-Linear Operations

- Median filtering for color images removes noise in each channel separately

original impulse noise 3x3 median

Non-Linear Operations

- K-nearest neighbors smoothing also removes noise in each color channel separately

original

impulse noise

3x3 kNN k=6

Non-Linear Operations

- Variance based enhancement can be applied to each color component to sharpen image detail

original

3x3 wallis

6x6 wallis

Non-Linear Operations

- Performing histogram equalization on each channel separately often causes a hue shift

original

histogram equalization

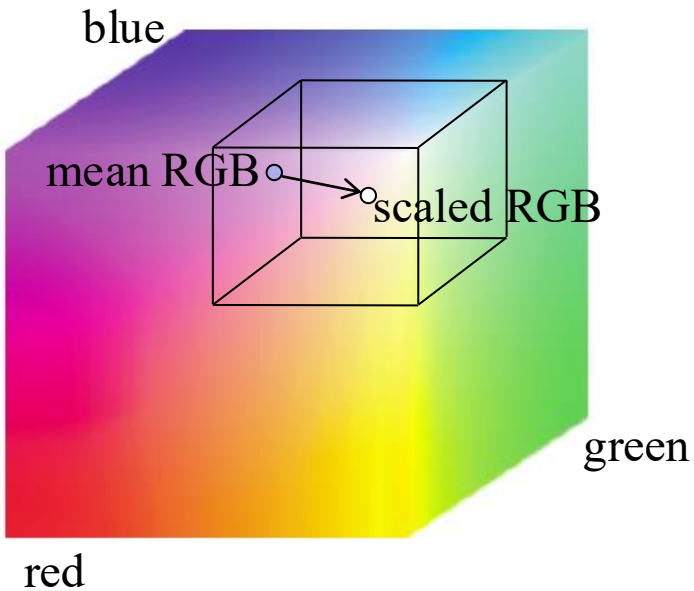
3x3 histogram equalization

Vector Based Processing

- Instead of processing color components in an image separately, we can treat colors as vectors in RGB space, and perform vector operations to manipulate the image
  - Advantage – keeps color components together so all three channels are changed in same time
  - Disadvantage – vector based methods are often more complex to implement

Vector Based Processing

- Image Whitening:
  - Calculate mean RGB of pixels > RGB midpoint
  - Calculate RGB scale factor to move mean RGB towards R=G=B line
  - Apply scale factor to all pixels to whiten image



Vector Based Processing

- Image whitening:

$$sumRGB = \sum_{\forall(x,y)} \begin{cases} RGB(x,y) & if\,(RGB) > midRGB \\ 0 & otherwise \end{cases}$$

$$countRGB = \sum_{\forall(x,y)} \begin{cases} 1 & if\,(RGB) > midRGB \\ 0 & otherwise \end{cases}$$

$$\overline{R}\,\overline{G}\,\overline{B} = sumRGB / countRGB$$

$$A = (\overline{R} + \overline{G} + \overline{B})/3$$

$$scaleRGB = (A, A, A)/(\overline{R}, \overline{G}, \overline{B})$$

Vector Based Processing

```
void im_color::Whiten()
{
    // Calculate current min max and mid values
    short MinR, MaxR, MinG, MaxG, MinB, MaxB;
    MinMax(MinR, MaxR, MinG, MaxG, MinB, MaxB);
    short MidR = (MinR + MaxR) / 2;
    short MidG = (MinG + MaxG) / 2;
    short MidB = (MinB + MaxB) / 2;

    // Initialize average white value
    float AveR = 0;
    float AveG = 0;
    float AveB = 0;
    int Count = 1;
```

Vector Based Processing

```
// Calculate average white value
for (int i = 0; i < R.NumPixels; i++)
if ((R.Data1D[i] > MidR) && (G.Data1D[i] > MidG) && (B.Data1D[i] > MidB))
{
        AveR += R.Data1D[i];
        AveG += G.Data1D[i];
        AveB += B.Data1D[i];
        Count++;
}
AveR /= Count;
AveG /= Count;
AveB /= Count;
```

Vector Based Processing

```
    // Whiten image
    float Ave = (AveR + AveG + AveB) / 3;
    float ScaleR = Ave / AveR;
    float ScaleG = Ave / AveG;
    float ScaleB = Ave / AveB;
    for (int i = 0; i < R.NumPixels; i++)
    {
        R.Data1D[i] = (short) (R.Data1D[i] * ScaleR);
        G.Data1D[i] = (short) (G.Data1D[i] * ScaleG);
        B.Data1D[i] = (short) (B.Data1D[i] * ScaleB);
        if (R.Data1D[i] > MaxR) R.Data1D[i] = MaxR;
        if (G.Data1D[i] > MaxG) G.Data1D[i] = MaxG;
        if (B.Data1D[i] > MaxB) B.Data1D[i] = MaxB;
    }
}
```

Vector Based Processing

- Original image had mean RGB of (195,161,66) so R is scaled down and B is scaled up to whiten image

Vector Based Processing

- The traditional way to calculate the median of N values $[X_1..X_N]$ is to sort the data and select X_{med} as the middle value of the sorted array

- Another way to find the median is to find the X_{med} value from $[X_1..X_N]$ where

$$\sum_{i=1}^{N} \left| X_{med} - X_i \right| \leq \sum_{i=1}^{N} \left| X_j - X_i \right| \quad \forall X_j \in [X_1..X_N]$$

Vector Based Processing

- Example:
  - Calculate median of [3 1 4 1 50]
  - Sorted array is [1 1 3 4 50]
  - Hence median value is 3

Vector Based Processing

- Example:
  - Calculate median of [3 1 4 1 50]
  - Distance from 3 to X_i is 0+2+1+2+47=52
  - Distance from 1 to X_i is 2+0+3+0+49=54
  - Distance from 4 to X_i is 1+3+0+3+46=53
  - Distance from 50 to X_i is 47+49+46+49+0=191
  - Hence median value is 3

Vector Based Processing

- The work required to calculate the median this way is $O(N^2)$, which more than the $O(N\log N)$ we could obtain by sorting the data

- Fortunately, the $O(N^2)$ approach extends easily to vector valued data $C_i=(R_i,G_i,B_i)$

$$\sum_{i=1}^{N} \left| C_{med} - C_i \right| \leq \sum_{i=1}^{N} \left| C_j - C_i \right| \quad \forall C_j \in [C_1..C_N]$$

Vector Based Processing

- When searching for the vector median C_{med} we can use either Euclidean distances (L2 norm) or absolute distances (L1 norm)

- "Vector Median Filters", J.Astola, P.Haavisto, Y.Nuevo, *Proc. IEEE*, Vol 78, No 4, 1990.

Vector Based Processing

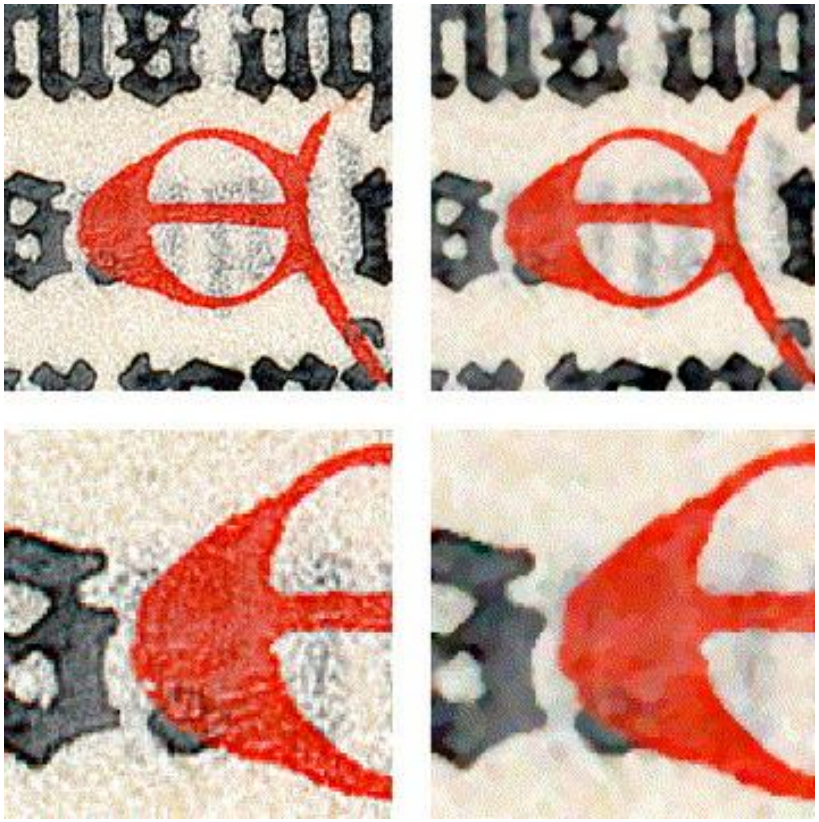
- Vector median filter applied to color image:

Vector Based Processing

- Vector median filter applied to color image:

Vector Based Processing

- Vector median filter applied to color image:



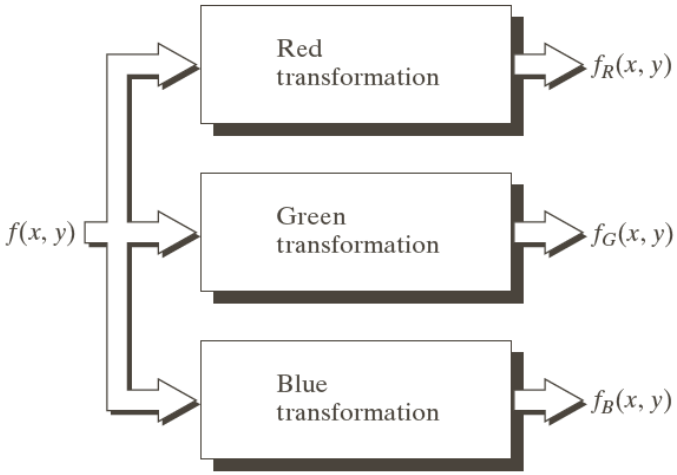
Pseudocolor Methods

- Pseudocolor methods typically transform a monochrome image into a color image

- Used to create images that are more easily viewed/understood by humans
  - Making it easier to see small pixel differences
  - Using colors that correspond naturally to data

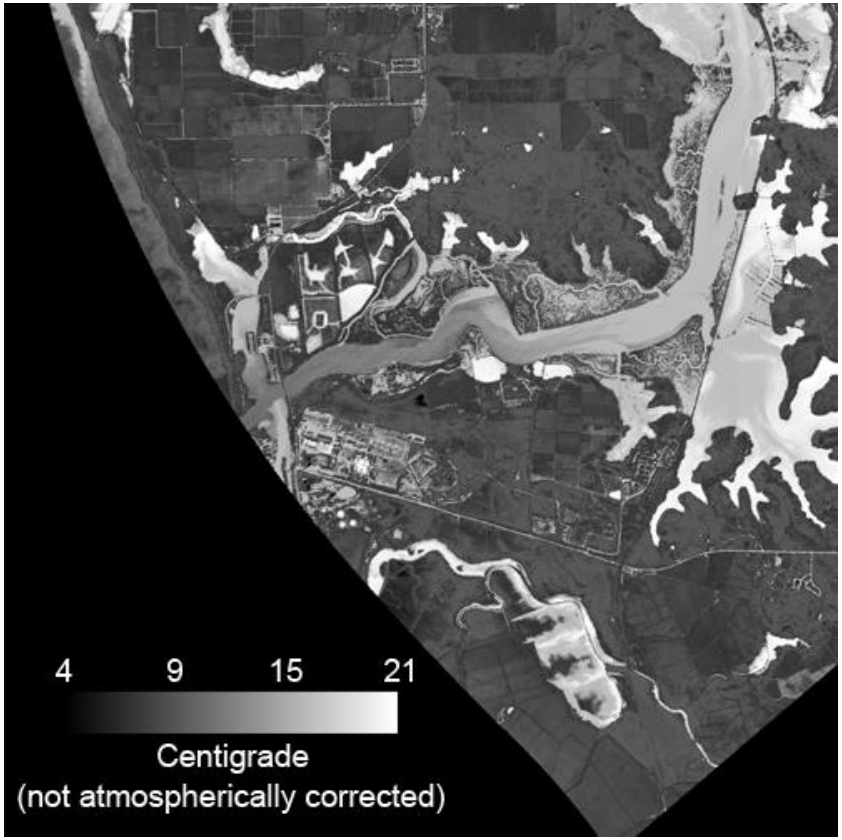
Pseudocolor Methods

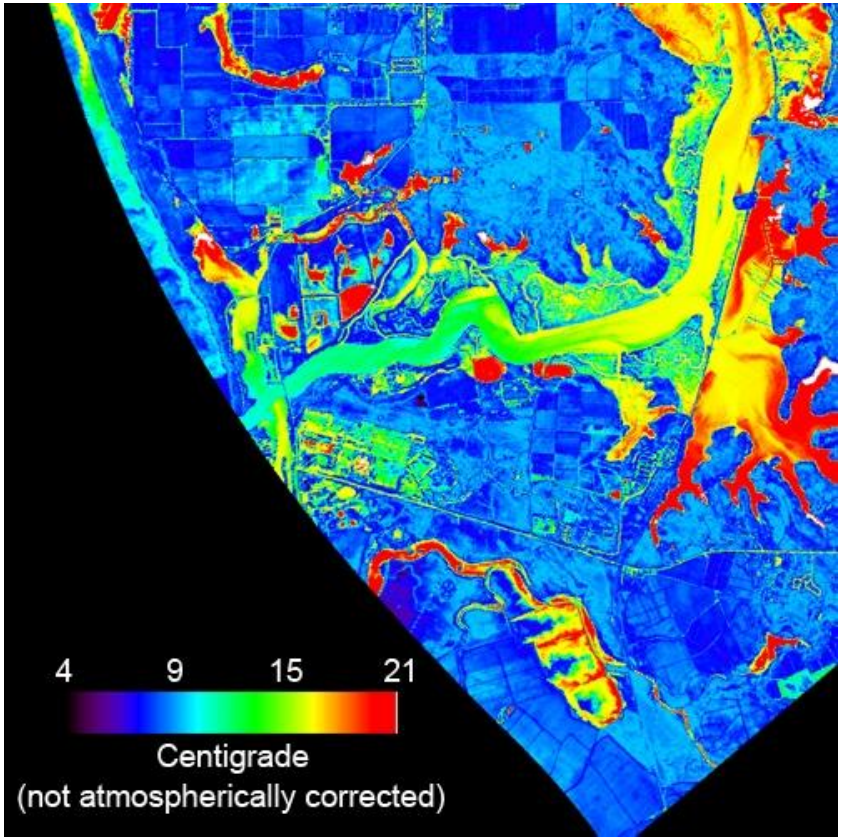
- Key is selecting an appropriate color map to convert from grey level to RGB color
  - Rainbow color scale is popular choice
  - Many other color scales are possible



Pseudocolor Methods

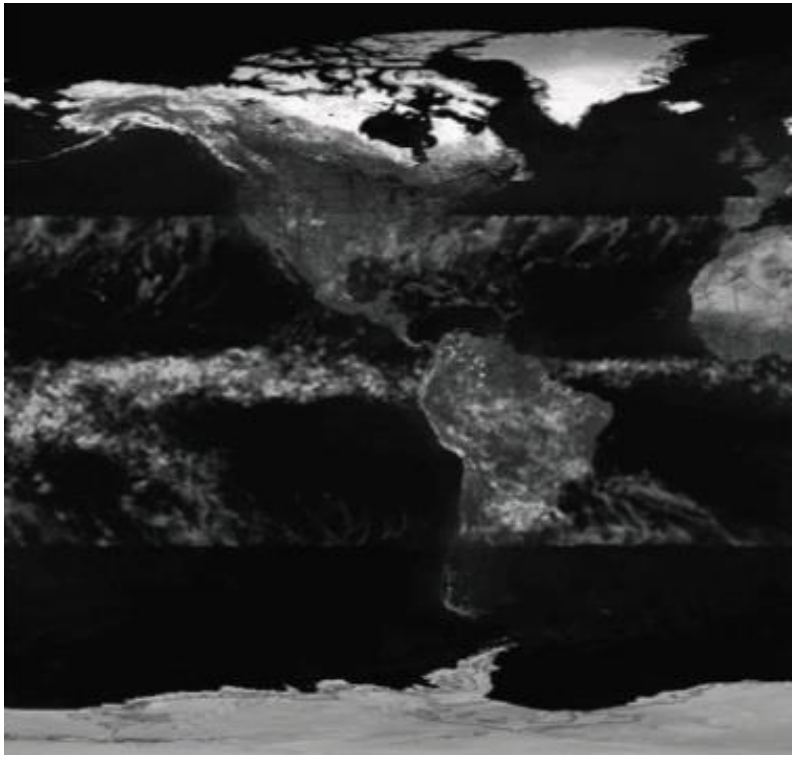
- Thermal image displayed using rainbow scale

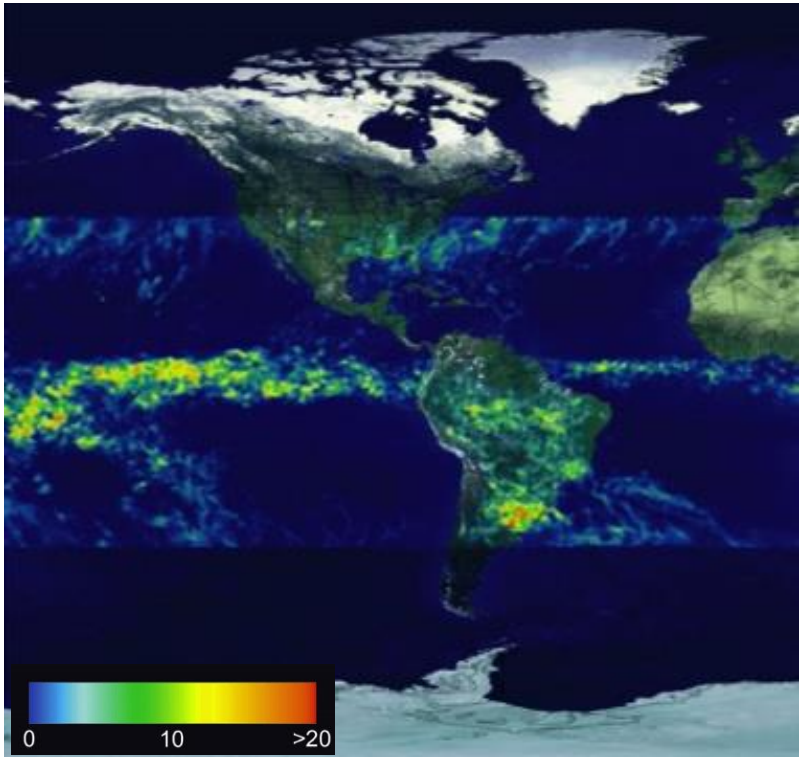




Pseudocolor Methods

- Rainfall image displayed using rainbow scale





Pseudocolor Methods

- Color image remapped to rainbow scale

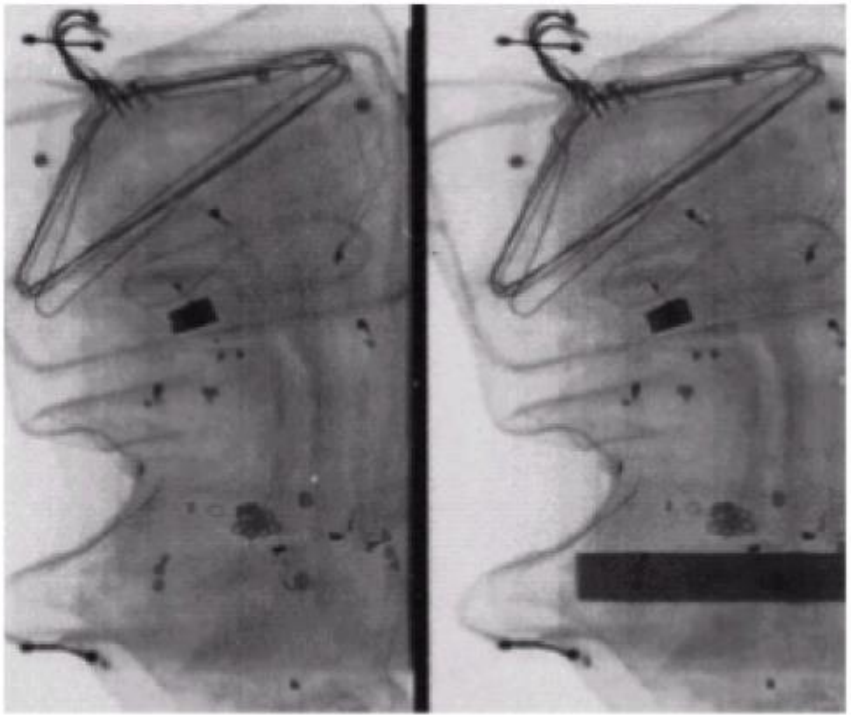
Pseudocolor Methods

- Some pseudocolor scales are designed to be *periodic*, with the same output colors for grey levels [0..N] as [N..2N] and [2N..3N] etc

- Pro - better visual separation of pixel values
- Con - duplication of output colors in image

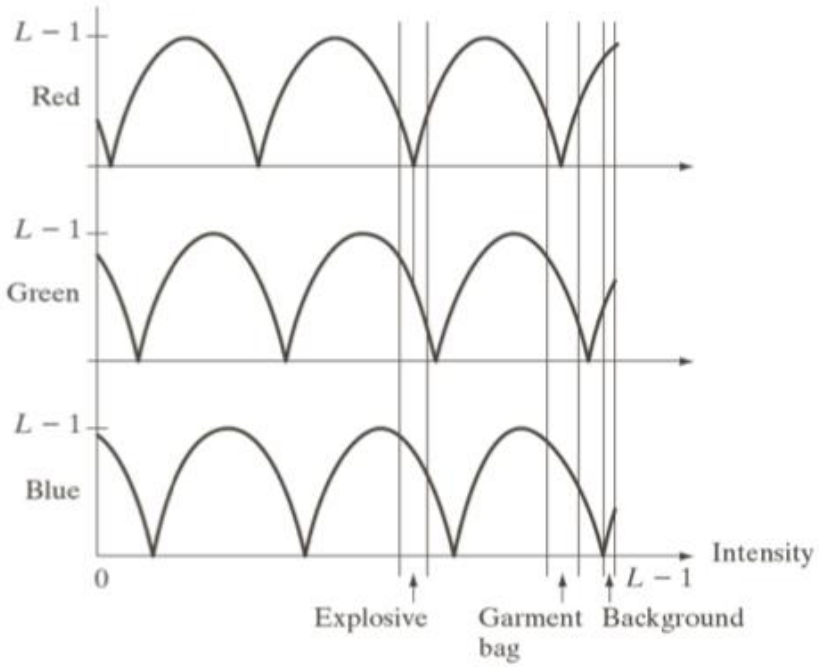
Pseudocolor Methods

- Baggage inspection:
  - Goal is to look at x-rays of bags and identify objects of interest
  - Large dynamic range of objects makes it hard to see inside some items

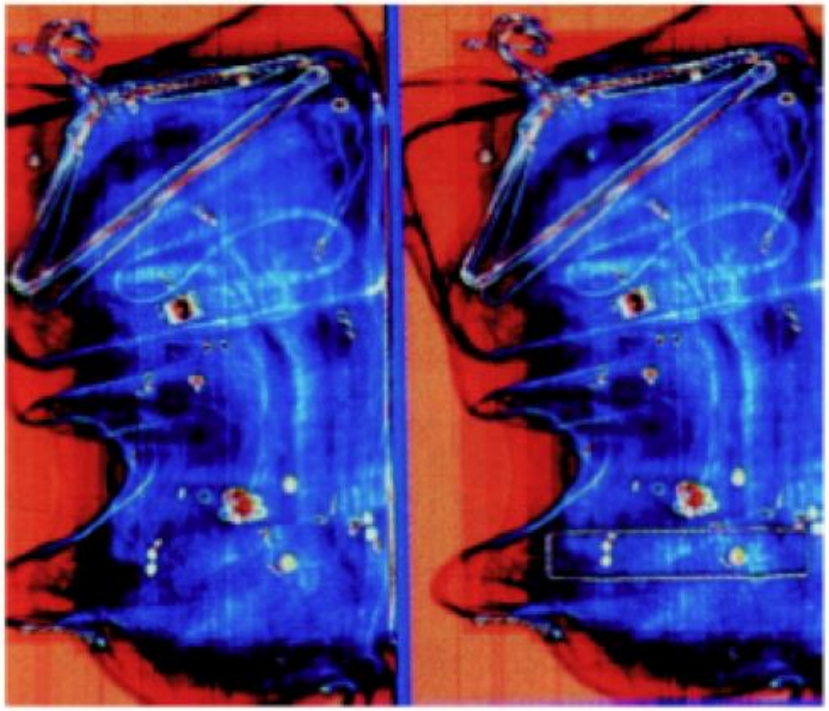


Greyscale image of bags

Pseudocolor Methods

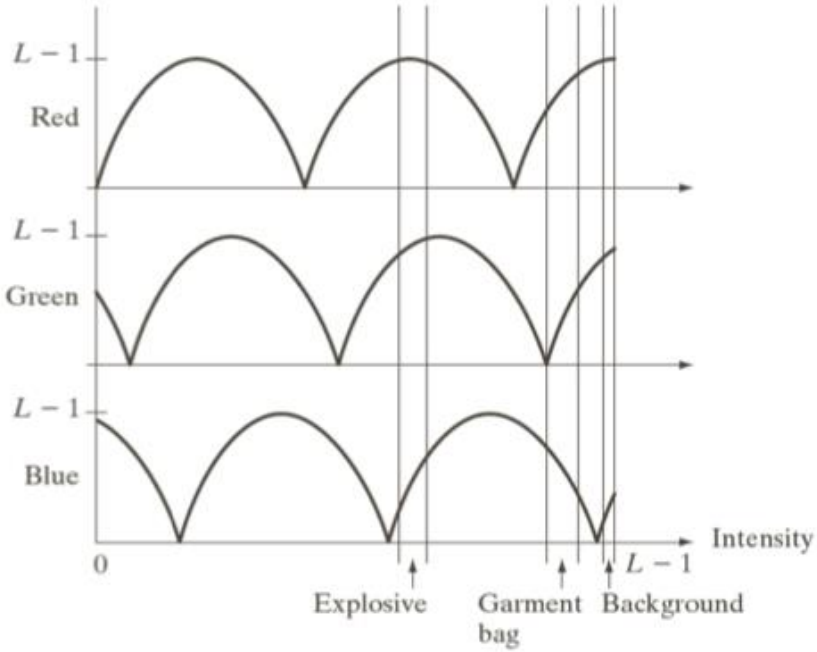


Periodic color scale

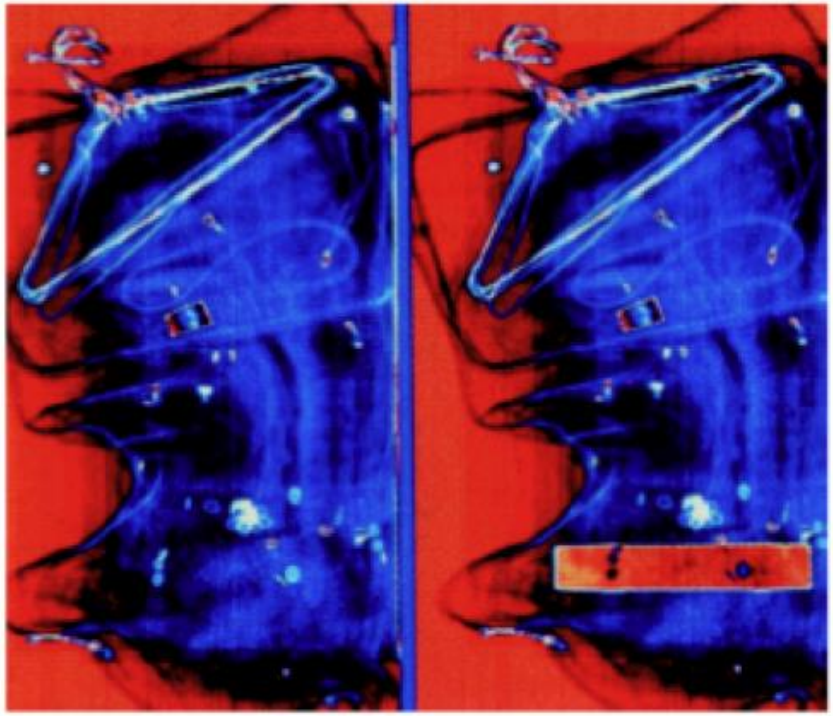


Pseudocolor image of bags

Pseudocolor Methods



Periodic color scale



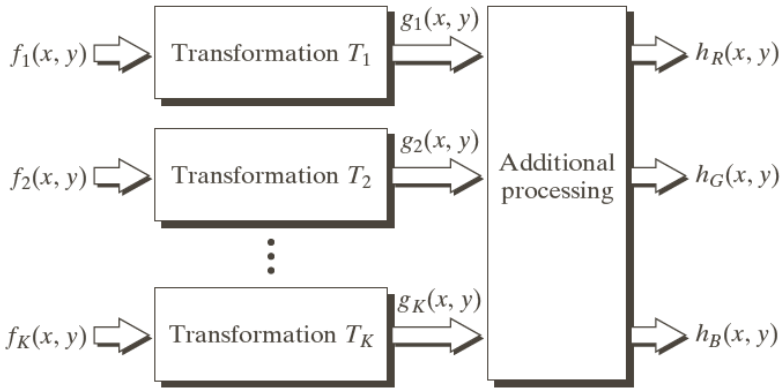
Pseudocolor image of bags

Pseudocolor Methods

- Multispectral satellites capture images in a wide range of spectral bands:
  - **Blue**, 450-520 nm
  - **Green**, 515-600 nm
  - **Red**, 600-690 nm
  - **Near infrared**, 750-900 nm
  - **Mid-infrared**, 1550-1750 nm
  - **Far-infrared**, 2080-2350 nm
  - **Thermal infrared**, 10400-12500 nm

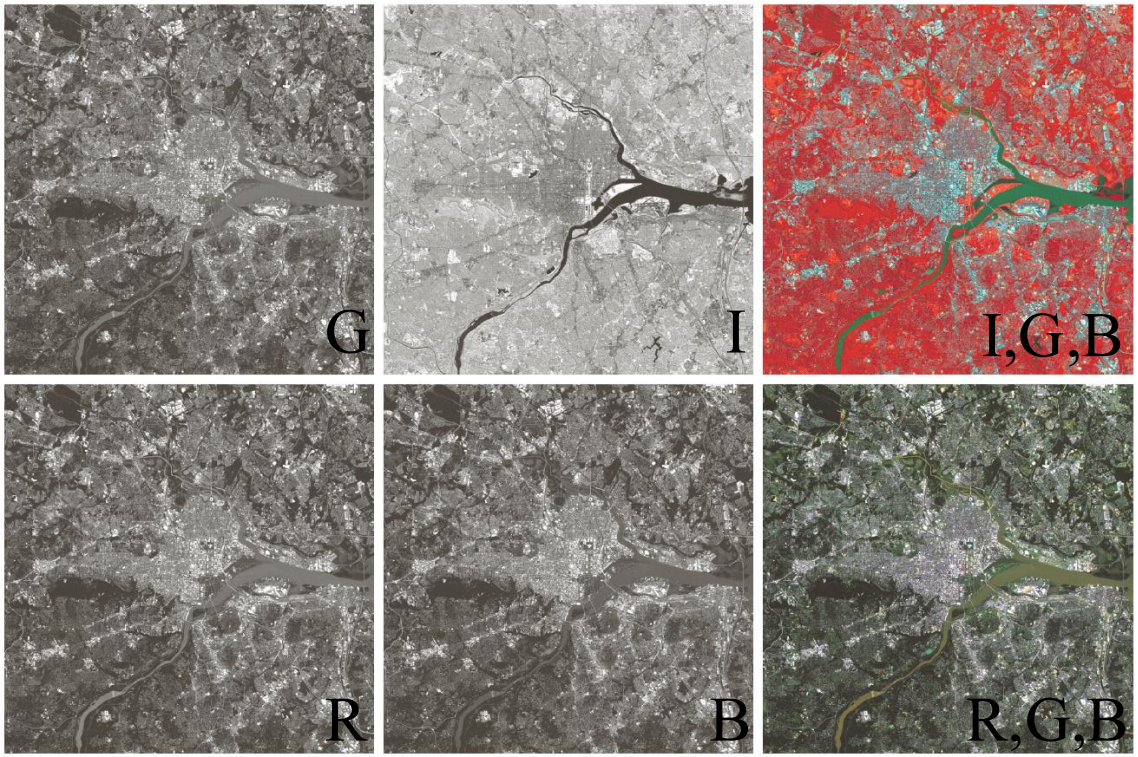
Pseudocolor Methods

- We can combine multispectral bands to create pseudocolor images for display purposes
- Key is deciding what bands to include and how they should be combined to visualize data



Pseudocolor Methods

- One option is to assign three bands to RBG channels to create pseudocolor images



Conclusion

- We have seen that color images can be converted into a wide range of color spaces with different advantages/disadvantages

- We have also seen that color images can be processed in several ways:
  - on a component-by-component basis
  - as vectors in a 3D space
  - using pseudocolor techniques